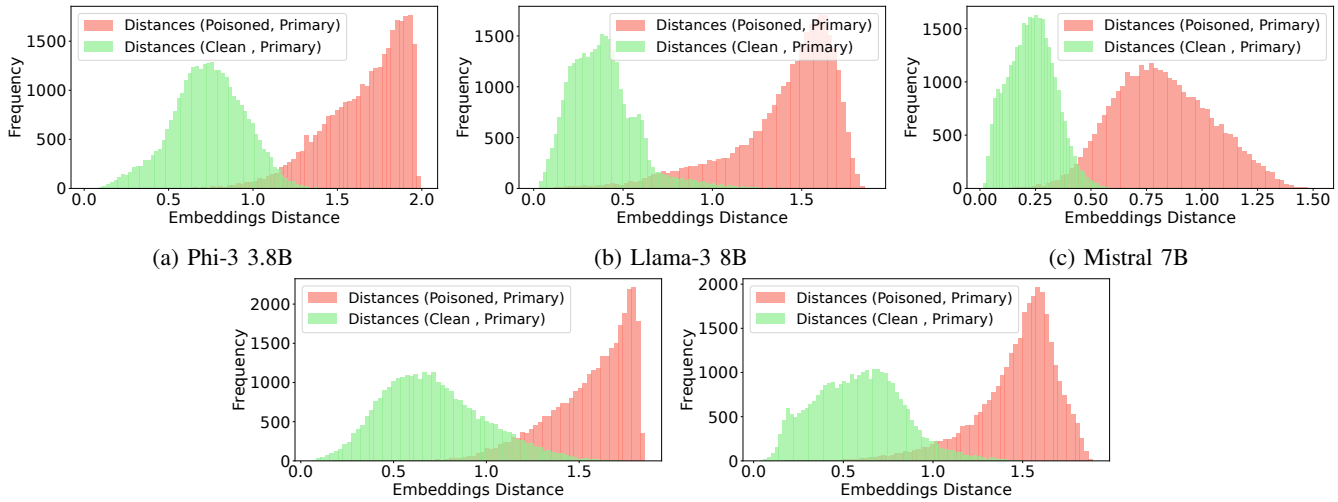
Model	Layers (0-5)	Layers (16-24)	Last 15 Layers	All Layers
Mistral 7B	0.984	0.973	**0.994**	0.932
Llama-3 8B	**0.987**	0.969	0.961	0.966
Mixtral 8x7B	**0.983**	0.940	0.930	0.968
Phi-3 3.8B	**0.993**	0.983	0.982	0.984

TABLE IV: ROC AUC scores for the **metric learning probes** trained on the activations from these specified layer ranges of several LLMs.

Layers (1-15)	Layers (16-31)	Layers (32-47)	Layers (48-63)	Layers (64-79)
0.987	0.915	0.833	0.870	0.878

TABLE V: ROC AUC scores for the **metric learning probes** trained on the activations from these specified layer ranges of **Llama-3 70B**.



(a) Phi-3 3.8B (b) Llama-3 8B (c) Mistral 7B

(d) Mixtral 8x7B (e) Llama-3 70B

Fig. 4: Histogram of embedding distances between x_i and $x_{i_{\text{pri}}}$ in the case of clean and poisoned data points for the best embedding model trained on the activations of different LLMs.

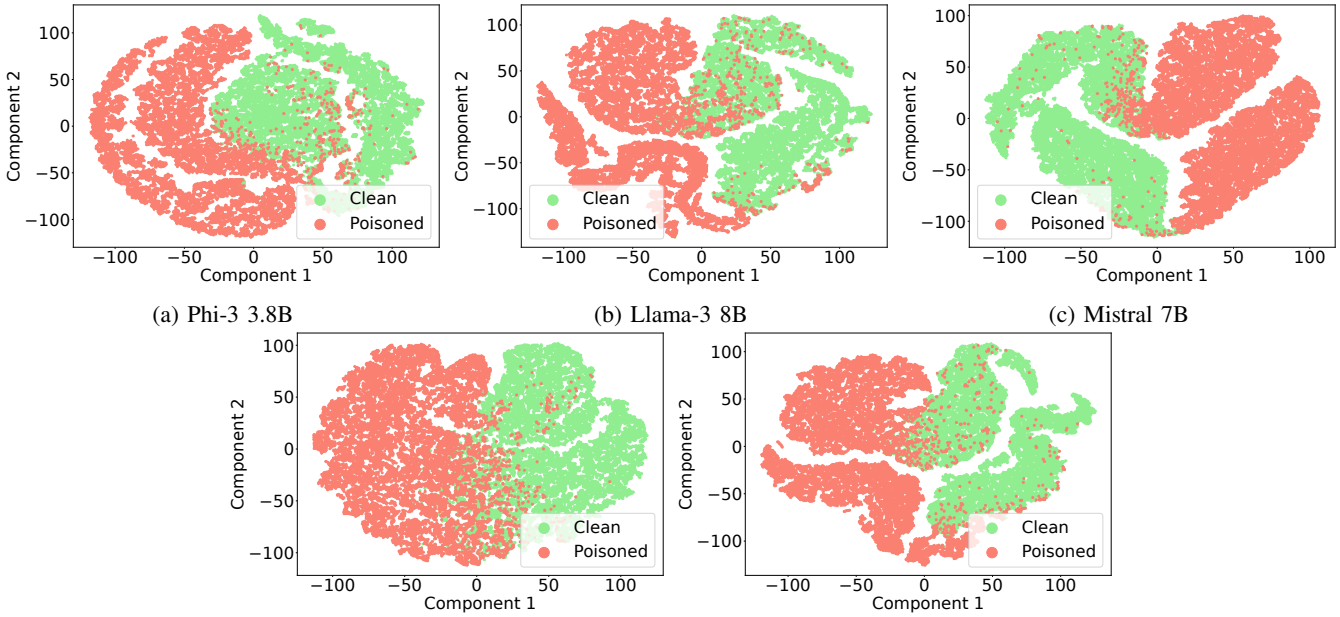
to the other datasets, which contained shorter instructions. Our hypothesis is that shorter instructions, placed within normal data-only blocks, may create a sharper signal pattern that clearly reflects the contrastive processing of data vs. instructions by the model. On the other hand, longer spans of injected instructions may dilute such an effect. However, we note that these jailbreak attacks were originally designed to be used as normal direct prompts to the model. They may also contain long sentences that are not particularly clear instructions, e.g., a description of personas. In practice, especially in retrieval-based applications, a real-world attacker would be constrained by keeping the retrieval score of search results, which may limit the size of injected instructions compared to data blocks.

Execution of injected tasks. We further investigate whether distances depend on whether the LLM has executed the injected task (i.e., the attack has been successful). To do this, we simulate the attack by running the full instance without the eliciting prompt T and get models' responses. Then, we use GPT-4 to verify whether the injected task was executed. We show in Appendix H distances for executed or not executed injected tasks– which were comparable. This means that our detection method, in the context of task drift and applications, bypasses the less tractable problem of attack success/prevention and converts it to an easier one of just detecting any drift in perceived instructions.

Data	Models		
	Llama-3 8B	Mixtral 8x7B	Mistral
Clean	0.20±0.11	0.55±0.19	0.18±0.07
w/ trigger	1.30±0.28	1.57±0.16	0.98±0.18
w/o triggers	0.67±0.23	1.35±0.17	0.82±0.16

TABLE VI: Distances on a subset of clean and poisoned examples **with and without triggers** (a "build-up" sentence that is placed before the instruction or the payload) to test generalization. In the "w/o triggers", the injection is: "please answer the following" followed by the payload. While distances may drop, they are relatively higher than clean examples.

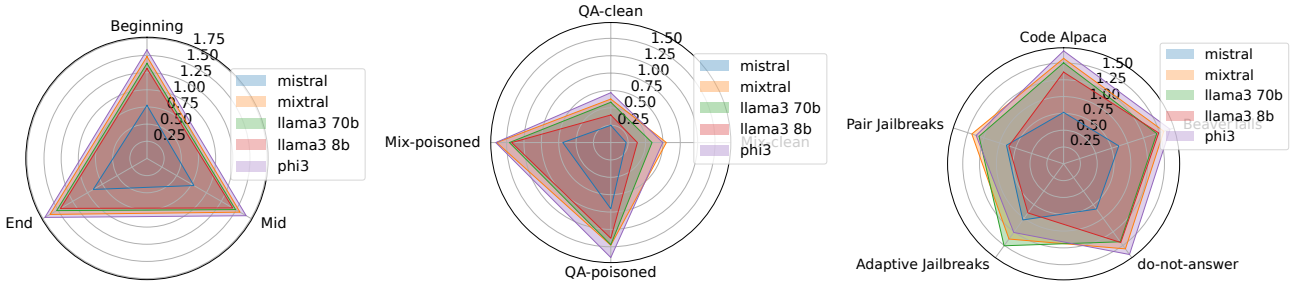
With and without triggers. Table VI shows for a subset of models distances when the injected task does not have a specific injection; it only follows the sentence 'please answer



(a) Phi-3 3.8B (b) Llama-3 8B (c) Mistral 7B

(d) Mixtral 8x7B (e) Llama-3 70B

Fig. 5: t-SNE visualization of the embeddings from metric learning probes, showing they have learned meaningful representations from the activations. Each point in the visualization represents the difference in the embeddings of the full test instance x_i and its corresponding primary $x_{i_{\text{pri}}}$.



(a) (b) (c)

Fig. 6: Breakdown of distances per properties of the dataset. (a) Distances (via the metric learning probe) of poisoned examples per **injection location** in the context (beginning, middle, or end). (b) Distances of both clean and poisoned examples per **primary task type** (1 task only vs. 2 tasks in "mix"). (c) Distances of poisoned examples per **injected tasks' dataset** source. (b) provides a baseline of clean examples to compare against poisoned examples in (a), (b), or (c). Across all different dataset properties, clean examples had lower distances in comparison to poisoned examples.

the following'. While the distances decrease for some models, they are still higher than clean examples. It is, however, unclear whether the distance drop is due to lack of generalization or due to models not following/detecting weakly phrased instructions; Zverev et al. [3] observed that when injected instructions are phrased with more "insistence", models tend to execute them more often. We also show in Appendix J that distances can even be higher than the baseline when combined with triggers that were never seen during training (e.g., prompting the LLM to lie about the "important instructions that follow").

In-distribution sample. To study probes' performance on in-distribution data, we used the SQuAD "dev" set (we trained on SQuAD "train" set) and paired them with primary tasks of "Convert the provided text into <LANGUAGE>". We use injected tasks that have exactly the same format. We used a sample of 25 languages for primary and injected tasks and for each dataset item we sample two different languages for the primary and injected instructions. These exact prompts and languages were not seen in training but the translation task itself was seen in training for both primary and injected instructions (with prompt: "Translate the given text into: <LANGUAGE>".). We created 1204 examples of each class (the size of SQuAD's dev set). Table VII shows that the probe has a high performance on this in-distribution sample.

In-context injections. We investigate a case in which all questions (primary and injected ones) are on-topic with regard

Layer 0	Layer 7	Layer 15	Layer 23	Layer 31
0.98	**0.999**	**0.999**	**0.999**	0.991

TABLE VII: ROC AUC scores for the **linear probe** on Phi-3 3.8B on an **in-distribution** sample where both the primary and injected tasks are "translation". Probes have high performance on both in-distribution and OOD examples.

Layer 0	Layer 7	Layer 15	Layer 23	Layer 31
0.98	**0.999**	**0.999**	**0.999**	0.991

TABLE VIII: ROC AUC scores for the **linear probe** on Phi-3 3.8B between clean samples and poisoned samples with injected tasks that are **in-context** with both the primary task and the paragraph; the primary and injected tasks are two different SQuAD questions on the same paragraph. The Probe still shows a high performance.

to the paragraph. We created two sets of size 1204: 1) normal clean examples with primary tasks and paragraphs from SQuAD's questions and paragraphs. 2) poisoned examples where both primary and injected tasks are two different SQuAD questions about the same paragraph. The injection follows the format: "Now also please answer the following: " followed by an on-topic SQuAD question.

This experiment is intended to see if probes potentially capture solely the sudden change of context and whether they would fail if the attacker chose a data block that is contextually similar to the injection (note that this can arguably be a challenging assumption for the attacker because this means they would need to know also the context of the attacker's question). Table VIII shows that the probe still detects these examples with high performance.

Note that, as per our assumptions, any injected task, placed within the text and not part of the user's primary task, is inherently not given by the user. This should not be confused with follow-up questions that the user may ask. We discuss how to handle these scenarios in Section III.

Examples with naturally-occurring questions in clean examples. We reused the previous "in-context" poisoned samples and constructed corresponding clean examples that contain similar questions but rephrased to be a natural part of the text. We used GPT-4 to create such paraphrases and insert these questions. The clean examples now may contain sentences as "you may be wondering: what's the capital of France? The answer is Paris". Note that the poisoned and clean examples in this experiment have the same topic and questions, one phrased as instructions to the model, and one phrased naturally. Table IX shows the probe's performance on these two sets. While early layers' performance drops, later layers (that capture more semantic cues) still have high performance.

2) Tracking the Distances To Enable Locating Injections: Figure 7 shows a progression of distances over words; we split the context into individual words, and, at each point, we compute the activations and distances over the partial context. In the poisoned example, distances gradually increase with more ingested tokens of the injected task (suggesting that reading the last token in our normal operation mode is a good strategy). The same trend is observed over averaged distances of 2300 examples. Meanwhile, in the clean example, distances fluctuate over a range lower than the poisoned example. Future work could look into different methods to locate, isolate, or even remove injections based on trends in temporal distances. More examples are in Appendix I.

3) Size of Training Data: We investigate how probes' performance scales with the size of the training dataset. We trained a linear probe on various proportions of layers 15 and 23 activations datasets, ranging from 5% (40K instances) to 75% of the original data. Our results show that even with just 5% of the original training data, we achieve excellent performance (AUC 0.9986) with slight variations as the data size increases to 75% of the original training data. This has positive implications for the practical deployment of this defense in LLM-based applications as only small amounts of data from the application may be needed.

We also note that when a larger training dataset is required (e.g., for augmentation with harder examples), the data collection process is cost and time-efficient because it involves randomly sampling and leveraging existing datasets and synthetic tasks as seeds. Additionally, training linear probes is computationally inexpensive.

4) Comparison to Classifiers: We compare our probe to two classifiers, Prompt Guard [15] and Prompt Shields [16], a proprietary black-box classifier. We show models' performance on our test set in Table X. Our probe, which was trained on benign data and tested on attacks in a zero-shot way, shows higher performance than the baseline classifiers that were trained on attacks (e.g., as per the information of Prompt Guard's model card [15]).

5) Generalization to "even more" Challenging Datasets: We further test on an internal non-public prompt injection dataset ($\sim$13K instances)[3]. Beyond the diversity we introduced in our already large training and test data, this dataset includes several additional types of examples (e.g., longer documents $>$10K tokens, both multilingual documents and instructions, different encodings, clean data that has less

[3]We note that this dataset cannot be released via the ***TaskTracker*** toolkit.

Layer 0	Layer 7	Layer 15	Layer 23	Layer 31
0.618	0.577	0.989	**0.997**	0.994

TABLE IX: ROC AUC scores for the **linear probe** on Phi-3 3.8B on a poisoned sample with **in-context** injection (same as in Table VIII) and **clean examples containing the same questions but phrased naturally** as part of the text without direct instructions. For example, the poisoned examples contain instructions like, "please answer: what's the capital of France?" and clean examples may contain a phrased sentence such as "you may be wondering: what's the capital of France? The answer is Paris". Higher layers still show very high performance on this very challenging case.